* Parallel programming is the technique of dividing a large problem into smaller tasks that run simultaneously on multiple processing units.
→ Reduce execution time.
→ Solve larger problems.
→ Improves resource utilization.

* multiple processors or cores work together, to communicate & coordinate to solve a computational problem simultaneously.

Key components of a parallel programming model.

1.) Task Decomposition.
2.) Communication.
3.) Synchronization.
4.) Mapping. (Assigning tasks to processors or cores).

Types of Parallelism

1) Task-level
2) Data-level
3) Instruction-level

Parallel programming approaches.

→ Message Passing Interface (MPI) ⇒ This allows processes to communicate by sending & recieving messages.

→ Open-MP ⇒ Used in multi-core processors.

→ CUDA (NVIDIA's GPU computing platform)
* It lets developers to use GPUs parallel processing capabilities.

→ MAP-Reduce (Hadoop popularized technique)
* This is designed for processing massive amounts of data in parallel

Advantages of parallel computing

→ Faster problem Solving.
→ Ability to handle big data.
→ Supports complex Scientific & Engineering Applications.
→ Improve real-time processing.
→ Better utilization of modern Hardware.
→ Energy efficiency.
→ Scalability.
→ Foundation for emerging technologies.

Shared memory

*This is a parallel model where multiple threads run on different processors but share a common global memory.

*Tasks share a common memory space that they read & write asynchronously.

Distributed Memory

* Processors have their own distinct memory.
* This model is prevelant in clusters & supercomputers.

Hybrid Architecture

* This combines both shared & distributed memory.

Types of Parallelism

1) Task-Level Parallelism
   → Breaking down a problem into different tasks.
2) Data-level Parallelism
   → This is concerned with processing numerous data components at the same time.
3) Instruction-Level Parallelism
   * To execute several instructions in a single processor cycle.

Race Condition in shared memory

* Two or more threads/processes access shared data at the same time & the final result depends on the unpredictable order of execution.

* Race condition is avoided by using such as;
→ Critical sections
→ Mutex locks
→ Atomic operations.
* Synchronization ensures correct program execution by preventing race conditions.

OPEN MP

* This is a shared-memory parallel programming API that uses compiler directives to create & manage threads
* Compiler directives (pragmas) enable parallelism.

FORK-JOIN Execution Model

* This is followed by OPen MP.

Steps

1) Serial Region
   * One master thread runs i.e, single thread runs program.

2) Fork
   * master creates a team of threads i.e; threads created.

3) Parallel Region
   * Work is divided among threads.

4) Join

* Threads merge back into single master i.e; threads terminate, return to single thread.

MPI (Message Passing Interface)

* This is a standard technique for distributed - memory parallel programming.
* Communicates through;
  → MPI - Send
  → MPI - Recv

Core Functions

1) MPI - Init()
2) MPI - Send () / MPI - Recv ()
3) MPI - Barrier ()
4) MPI - Reduce()

Feature	Shared Memory	Distributed memory
Memory	Common	Private per node.
Communication	Through Variables	Message Passing.
Scalability	Limited	Very high
Speed	Faster local access	Slower due to network.
Example	Multi-core CPU	Cluster / Super computer.
Programming tools	OpenMP	MPI.

Hybrid Programming (MPI + OpenMP)

* Combines MPI b/wn nodes & OpenMP within a node.
→ MPI handles inter-node communication
→ OpenMP handles intra-node threading.

Advantages

→ Better Scalability.
→ Reduced Communication.
→ Efficient Resource use.
* Used in modern HPC systems.

Numpy in Scientific Computing

* Core library for Numerical & scientific computing in Python.

Key features

→ ndarray
→ Vectorization
→ Broadcasting
→ Optimized performance.

Applications

→ Data Analysis.
→ Machine learning.
→ Image processing.
→ Scientific simulations.

NumPy ndarray

* Homogeneous data type
* Faster than python lists

Broadcasting Rules

→ Compare shapes from right to left.

→ Dimensions must be equal or one must be 1.

* Eliminates explicit loops.

Scipy (Scientific python).

* This is scientific computing library built on NumPy that provides advanced mathematical & scientific algorithms.

→ scipy.linalg (Linear Algebra).
→ scipy.optimize
→ scipy.integrate
→ scipy.stats
→ scipy.sparse

*Linear Algebra Operations

→ Solving equations.
→ Eigenvalues
→ Matrix inversion
→ Decompositions.

Disk I/O

* This refers to reading & writing data to persistent storage such as HDD's & SSD's.

Types

→ Blocking I/O
→ Non-Blocking I/O
→ Memory mapped I/O.

* Chunk-based reading & buffering optimization techniques are used.

Threading & Concurrency

* A thread is a lightweight unit of execution within a process.

Concurrency ⇒ Multiple tasks make progress at the same time.

Parallelism ⇒ Tasks execute simultaneously.

Global Interpreter Lock

* This ensures only one thread executes Python byte code at a time.
* Simplifies memory management & prevents race conditions.

* Does not affect I/O bound tasks.

→ Numpy & Scipy release GIL internally

POSIX Threads

* These are standardized "C" API for multithreading & concurrent programming.
* Provides true parallelism.
* Low-level thread control.
* Explicit Synchronization.

Applications

Numpy

→ Imagine pixel manipulation.
→ Feature Scaling.
→ Matrix Computations.

Scipy

→ Signal filtering
→ Statistical modeling.
→ Optimization problems.

Threading

→ File downloads
→ Network servers.
→ Background tasks.

Synchronization Techniques

Feature	mutex	Semaphore	Barrier
Purpose	Mutual exclusion	Resource control	Thread coordination
Counter	No	Yes	No
Ownership	Yes	No	No
Blocking	yes	yes	yes
Typical use	Critical section	Resource sharing	Phase synchronization.

Synchronization Issues

→ Deadlock
→ starvation.
→ Priority Inversion.

Scheduling

* O.S decides which task runs on which processor and for how long.

→ Preemptive scheduling
→ Priority scheduling.
→ Load Balancing
→ Parallel scheduling.

High Performance Tasks

→ Compute-intensive.
→ Data parallel.
→ Latency sensitive or throughput-oriented.
→ Parallel execution & memory bandwidth optimization.

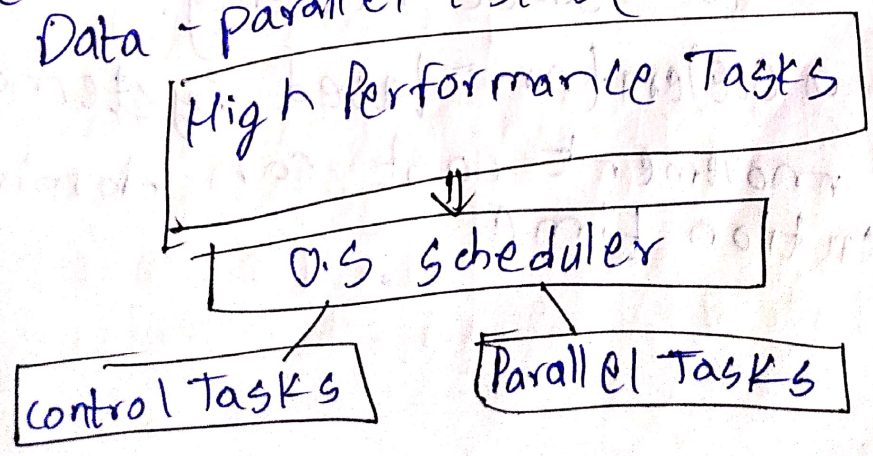
Importance

→ Prevents idle accelerators.
→ Reduces execution time.
→ Improves throughput.
→ Supports real-time deadlines.

Types

1) Task placement Scheduling

*Scheduler classifies tasks based on computational characterstics,
→ Control-intensive tasks (CPU)
→ Data-parallel tasks (GPU or accelerator).



2) Heterogeneous Scheduling

* This manages different types of processing units with varying performance characterstics.

* Decisions are based on,
  - → Task size
  - → Parallelism level
  - → Data transfer cost

3) Load Balancing

* No processing unit remains idle while others are overloaded.

4) Preemptive vs Non-Preemptive Scheduling

* Manages different types of processing units with varying performance

5) Data-Aware Scheduling

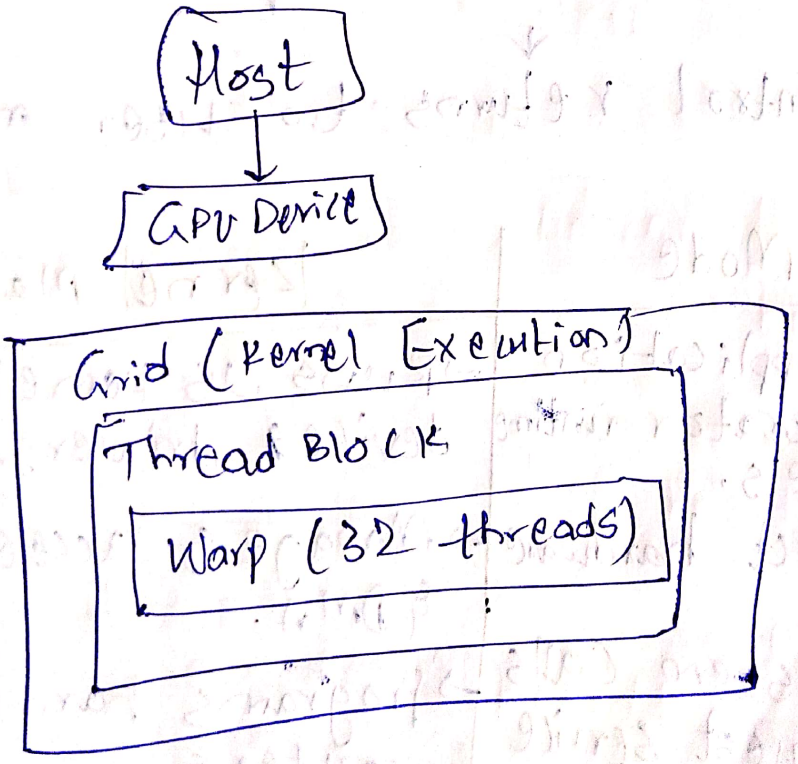
* Schedule tasks close to their data to reduce latency.
* In accelerator-based systems, data movement cost can dominate execution time.

GPU scheduling

→ GPU task execution applies classical scheduling concepts such as,
* queue-based, non-preemptive, time-sliced execution & data aware scheduling.

GPU Execution model

→ Kernel = A GPU task.
→ Grid = Collection of thread blocks
→ Thread Block = Scheduled unit on GPU.
→ Warp = Basic scheduling unit (32 threads)



Advantages

→ Extremely low scheduling overhead.
→ Efficient latency hiding.
→ High hardware utilization.

System Call

* System calls form the interface between applications & hardware

Working

User program invokes a system call

↓

Trap instruction is executed

↓

CPU switches from user mode to kernel mode

↓

Kernel executes requested service

↓

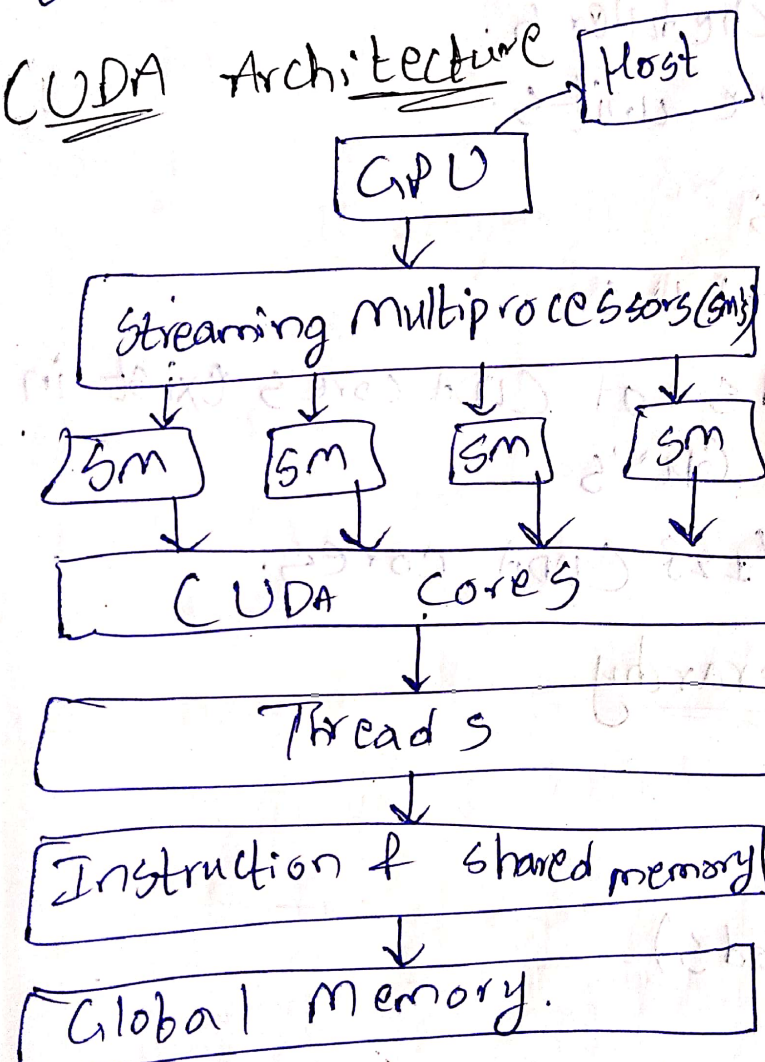
Control returns to user mode

User Mode	Kernel mode
→ Runs application & accelerator runtime libraries.	→ Runs OS kernel & device drivers.
→ No direct hardware access.	→ Manages accelerators & DMA.
→ Uses system calls to request service	→ Programs hardware registers.
→ Failures are contained	→ Handles interrupts from accelerators.

CUDA (Compute Unified Device Architecture)

* This allows multiple threads to execute simultaneously.
* We use CUDA for massive parallelism, High performance & use of development tools like cuBLAS, cuDNN.

CUDA Architecture



Host

* Executes serial part of program.
* Connected to GPU via PCIe bus.

Device (GPU)

* Executes massively parallel Kernels.
* Contains multiple SMs.

Streaming multiprocessors

* This is core processing unit of GPU
* Each SM contains :-
  - → CUDA cores
  - → Registers
  - → Shared memory.
  - → Warp schedulers.
  - → Load/Store units.

CUDA cores

- → Simple ALU's.
* Thousands of CUDA cores exist in modern GPU's.

⇒ 1 SM ⇒ 64/128 CUDA cores.

Thread Hierarchy

Thread
⇓
Warp
(32 threads)
⇓
Block
(Group of threads)
⇓
Grid
(Collection of thread Blocks).

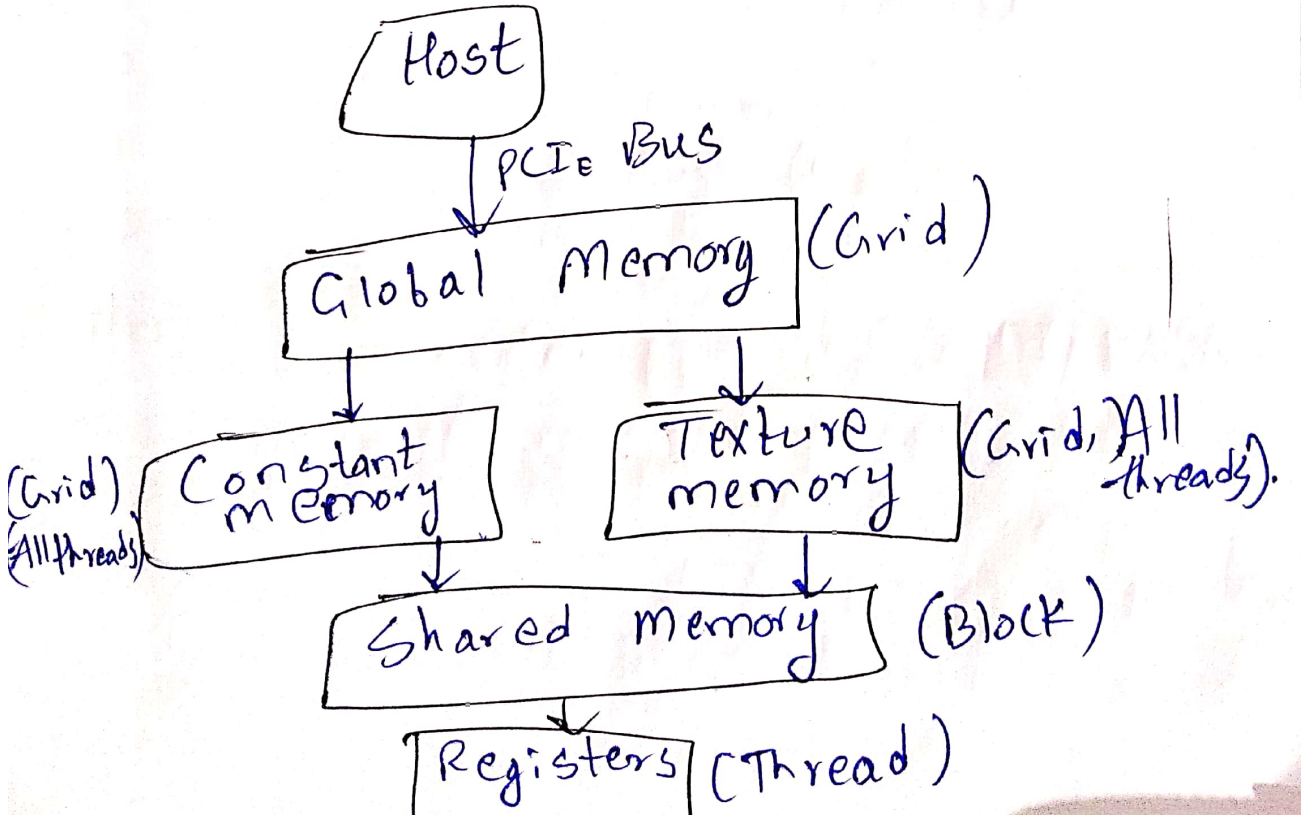
CUDA working

1.) Allocate memory on the GPU
2.) Copy data from CPU (host) to GPU
3.) CPU instructs the GPU to run a function in parallel.
4.) Copy results back from GPU to CPU.

Advantages

→ Mature Ecosystem & Libraries.
→ Superior performance.
→ Unified memory
→ Massive community support.
→ Advanced Hardware Integration.

CUDA memory Hierarchy



Advantages of memory hierarchy.

→ Reduced execution time.
→ Improved parallel efficiency.
→ Better scalability.
→ Optimal GPU utilization.

CUDA real-world Applications

→ Deep learning
→ Cryptography
→ Matrix math.